Cross-Layer KV-Cache Pruning for Low-Latency LLM Inference on Resource-Constrained Hardware: How Much Does Per-Input Adaptivity Buy?

Khush Patel Sneha Dash
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India
BCSE306L Natural Language Processing — Project Report — Faculty guide: Dr. Vijayaprabakaran K
Code and results: github.com/khushpatel3016/kv-cache-entropy-pruning

***Abstract*—The key–value (KV) cache of a decoder-only language model grows linearly with the prompt and, on small GPUs, quickly exceeds the memory taken by the weights. Pruning methods keep only part of it, and recent ones split a fixed memory budget unevenly across layers, either with a fixed schedule (PyramidKV) or adaptively for every input (CAKE, DynamicKV). We ask how much that per-input adaptivity actually buys, and whether attention entropy alone is a sufficient signal for it. We build a controlled framework in which every method shares one prefill, one token-selection rule and an identical total budget, so differences come only from the per-layer split, and we evaluate EntropyKV, an allocator driven by length-normalised attention entropy from a 32-query observation window, against seven baselines and controls. On Qwen2.5-3B (600 LongBench samples, budgets 10–50%), EntropyKV keeps 92% of full-cache accuracy at a 10% budget and beats PyramidKV by 5.5 points (95% CI 3.9–7.4), but it does not beat a uniform split: it ties SnapKV at 10% and 30% and trails it by 0.4–0.7 points at 20% and 50%. Recomputing the split per input gives no measurable gain over a frozen offline profile. Across 14 allocations, accuracy loss is explained by how little the most-starved layer receives (r = 0.75), not by the direction of the entropy signal; at an extreme 2% budget, reversing that direction recovers 24 points on passkey retrieval for Qwen, while Llama-3.2-3B is unaffected. Pruning cuts the 16K-token cache from 573 MB to 57 MB and decode latency from 139 to 44 ms per token on a T4, with an allocation overhead below 1% of prefill time. All code, 32,700 logged results and figures are public.**

***Index Terms*—large language models, KV cache compression, attention entropy, long-context inference, efficient inference, LongBench.**

I. Introduction

Autoregressive transformers [1] store a key and a value vector for every past token in every layer so that each new token can attend to the whole context without recomputing it. This *KV cache* is what makes decoding fast, but its size grows with the prompt: for a model with L layers, h_{kv} key–value heads of dimension d_{h} and a prompt of n tokens it holds $2{\cdot}L{\cdot}h_{\text{kv}}{\cdot}d_{\text{h}}{\cdot}n$ numbers. For Qwen2.5-3B in fp16 this is 36 KB per token, so a 16K-token prompt needs 573 MB of cache on top of 6 GB of weights, and every generated token must read all of it. On consumer and free cloud GPUs (a 16 GB T4, a 6 GB laptop card) the cache, not the arithmetic, sets the maximum context and the decoding speed.

KV-cache pruning keeps only the tokens that later queries are likely to need. Early methods such as H2O [4] and SnapKV [7] keep the same number of tokens in every layer. Later work observed that layers attend differently, and allocated the budget unevenly: PyramidKV [8] with a fixed schedule that gives lower layers more, and CAKE [10] and DynamicKV [9] adaptively, recomputing the split for every input from its attention pattern. Adaptive methods report gains, but they also change several things at once (signal, eviction score, memory management), so it is hard to tell how much of the gain comes from adapting the split to each input.

This project isolates that one factor. We ask: *how much does per-input adaptivity of the per-layer budget actually buy, and is attention entropy alone a sufficient signal to drive it?* We study it in the regime that motivates pruning most, small grouped-query-attention (GQA) [3] models on a free 16 GB GPU. Our contributions are:

- **A controlled comparison framework.** All methods share one prefill, one token-selection rule (SnapKV-style window scoring) and an exactly equal total budget; they differ only in how that budget is split across layers. A random-split control and a frozen-profile control are included.
- **EntropyKV**, a minimal single-signal allocator from length-normalised attention entropy over a 32-query observation window, and **a static-versus-dynamic ablation** that measures the value of recomputing the split per input.
- **An explanation of when allocations fail.** Across 14 allocations, loss is driven by starving the least-funded layer, not by the direction of the signal; at extreme budgets the direction matters, in a model-dependent way, which we confirm with a targeted reversal experiment.
- **A reproducible pipeline** on free hardware: 63 automated correctness tests, resumable two-GPU Kaggle runs, and 32,700 logged results across two model families, from which every figure in this report regenerates.

The answer is mostly negative for adaptivity and positive for the framework: per-input entropy allocation is safe and cheap and clearly better than a fixed pyramid, but on LongBench it does not beat a uniform split or a frozen profile. Section II reviews related work and Section III states the research questions. Sections IV–VI describe the method, implementation and setup, Section VII reports results, and Sections VIII–X discuss them, list limitations and conclude.

II. Related Work

A. Token eviction with a uniform budget

H2O [4] keeps “heavy-hitter” tokens with the largest accumulated attention plus a recent window; Scissorhands [5] relies on the same persistence of importance. StreamingLLM [6] keeps only a few initial “attention sink” tokens and a sliding window, which is cheap but forgets the

middle of the context. SnapKV [7] observes that the attention of the last few prompt tokens (an *observation window*) predicts which tokens generation will need, and selects tokens once, after prefill, by pooled window attention. All of these give every layer the same budget.

B. Per-layer budget allocation

PyramidKV [8] notes that attention becomes more concentrated in higher layers and assigns budgets that decrease linearly with depth, fixed for all inputs. DynamicKV [9] adjusts per-layer budgets for each input during prefill from its attention pattern (in our DA1 review we wrongly described it as adapting per task category only; Table I corrects this). CAKE [10] is the closest work to ours: it measures each layer's spatial attention dispersion with entropy and its temporal shift with variance, allocates budgets per input from their product, and adds cascading memory management so that prefill peak memory also shrinks. Ada-KV [11] adapts budgets across heads within a layer, and FastGen [12] chooses a compression policy per head from its attention profile.

C. Other directions

DuoAttention [13] and the retrieval-head analysis of Wu *et al.* [14] show that a small set of heads performs long-range retrieval while others attend locally; we return to this in Section VIII. Orthogonal techniques reduce bytes per token instead of tokens, by quantisation (KIVI [15]), or manage cache memory more efficiently (PagedAttention [2]); they can be combined with pruning.

TABLE I
PER-LAYER ALLOCATION IN KV-CACHE PRUNING METHODS

Method	Venue	Budget split across layers	Signal for the split	Per input?	Token selection
H2O [4]	NeurIPS'23	Uniform	—	—	Accumulated attention + recent
StreamingLLM [6]	ICLR'24	Uniform	—	—	Sink tokens + recent window
SnapKV [7]	NeurIPS'24	Uniform	—	—	Pooled observation-window attention
PyramidKV [8]	arXiv'24	Fixed linear decrease with depth	Layer index	No	SnapKV-style
DynamicKV [9]	EMNLP-F'25	Adaptive, recomputed during prefill	Attention pattern	Yes	Window attention
CAKE [10]	ICLR'25	Adaptive, cascading during prefill	Entropy × variance	Yes	Attention + variance indicator
Ada-KV [11]	arXiv'24	Across heads within a layer	Head attention mass	Yes	Top-k per head
This work	**—**	**Adaptive (and frozen, as ablation)**	**Normalised entropy only**	**Yes / No**	**SnapKV-style (shared by all methods)**

D. Research gap

Per-input allocation already exists, so this project does not claim it as new. What is missing is a *controlled* measurement of its value. Published adaptive methods change the allocation signal, the eviction score and the memory schedule together, compare against baselines implemented in different code bases, and do not include a random-allocation control or a frozen version of their own profile. It is therefore unclear (i) whether recomputing the split per input beats a fixed, data-derived profile, (ii) whether a single cheap signal suffices, and (iii) why some allocations hurt. These questions are also untested for the small GQA models (2–8 KV heads) that run on free hardware.

III. PROBLEM FORMULATION

Given a prompt of n tokens and a budget ratio ρ, every pruned method keeps exactly

$$B = \lfloor \rho \, L \, n \rfloor \tag{1}$$

tokens in total, split as $b_1, \ldots, b_L$ with $\Sigma b_l = B$ and $b_l \le n$. Methods differ only in this split. We pre-registered four research questions and three hypotheses before running the main experiments:

- **RQ1:** at matched budgets (10, 20, 30, 50%), does entropy-driven allocation beat a uniform split (SnapKV) and a fixed pyramid (PyramidKV)?
- **RQ2:** does recomputing the split per input beat a frozen, offline-averaged entropy profile?
- **RQ3:** does entropy alone come close to CAKE's entropy×variance rule?
- **RQ4:** what are the memory, latency and allocation-overhead costs?

H1: dynamic allocation beats fixed allocation at low budgets (10–20%), with the gap closing near 50%. **H2:** the dynamic-versus-static gap is largest on heterogeneous tasks. **H3:** entropy alone reaches at least 90% of CAKE's score. Section VIII states which held.

IV. METHOD

A. Overview

Fig. 1 shows the pipeline. The prompt is processed once (prefill). During prefill, a custom attention function computes, for every layer, attention statistics from an observation window without ever forming the $n \times n$ attention matrix. After prefill, the allocator turns those statistics into per-layer budgets, each layer keeps its own top-scoring tokens, and decoding proceeds with caches of different lengths per layer.

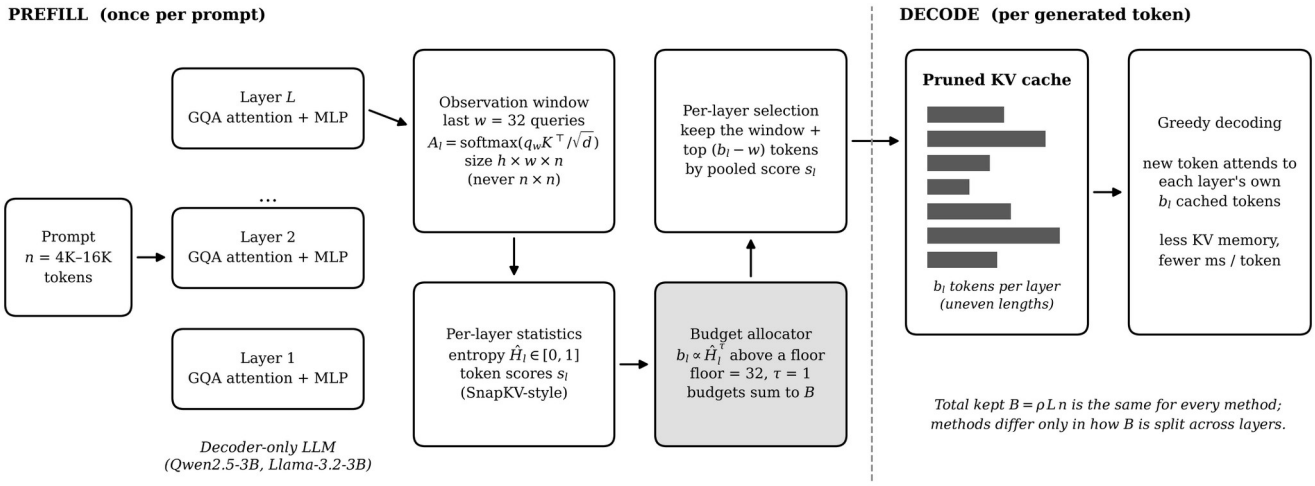


Fig. 1. Pipeline of the controlled framework. All methods share the prefill, the observation-window statistics and the token-selection rule; only the budget allocator (shaded) changes between methods. Pruning happens once, after prefill.

B. Observation-window statistics

Let Q_w be the last $w = 32$ query vectors of layer l (after rotary embedding) and K all n keys, with GQA query groups mapped to their shared KV head. We compute

$$A_l = \mathrm{softmax}\left(\frac{Q_w K^\top}{\sqrt{d}}\right) \in \mathbb{R}^{h \times w \times n} \tag{2}$$

with the causal mask applied inside the window. Each row is a distribution over the keys visible to that query. Its entropy is normalised by the logarithm of the number of visible keys n_i, so that prompts of different lengths are comparable, and averaged over heads and rows:

$$\hat{H}_l = \frac{1}{h\,w} \sum_{\text{heads}} \sum_{i=1}^{w} \frac{-\sum_j p_{ij} \log p_{ij}}{\log n_i} \in [0, 1] \tag{3}$$

High $\hat{H}_l$ means the layer spreads attention over many tokens; low $\hat{H}_l$ means it focuses on a few. The same pass also produces each token's score $s_{l,j}$, the attention it receives from the window summed over heads and rows, and, for the CAKE-style baseline only, the variance of attention across window rows. Memory is $O(h \cdot w \cdot n)$ per layer instead of $O(h \cdot n^2)$.

C. Entropy-driven allocation (EntropyKV)

Budgets are proportional to entropy above a floor f that prevents any layer from collapsing:

$$b_l = f + (B - Lf) \frac{\hat{H}_l^\tau}{\sum_{k=1}^{L} \hat{H}_k^\tau} \tag{4}$$

with $f = 32$ and $\tau = 1$ by default. $\tau = 0$ recovers the uniform split, $\tau > 1$ sharpens it, and $\tau < 0$ reverses it so that focused layers receive more. Budgets above n are capped and the overflow redistributed (water-filling), and largest-remainder rounding makes the integer budgets sum to exactly B. Spreading memory towards high-entropy layers follows the intuition behind PyramidKV and CAKE: a layer that attends broadly needs more tokens to cover what it uses.

D. Token selection

Inside each layer, every method keeps the w window tokens plus the $b_l - w$ prefix tokens with the highest scores after average pooling with kernel 7 (SnapKV-style window scoring [7]; our DA1 report described this step as H2O-style, but H2O's cumulative score is not available at prefill time). Keys and values of a layer's KV heads are pruned to the same token set.

E. Static profile and baselines in the same framework

EntropyKV-static uses the same allocator with a frozen profile: per-layer entropies averaged over 54 prompts from six *held-out* LongBench tasks (NarrativeQA, MultiFieldQA, 2WikiMQA, MultiNews, SAMSum, PassageCount) that never appear in evaluation. **SnapKV** uses equal budgets. **PyramidKV** uses its published linear schedule with $\beta = 20$ (the deepest layer gets 1/20 of the average). **Random allocation** splits the same total with Dirichlet(1) weights above the same floor; if entropy does no better than this, entropy is not the reason for any gain. **CAKE-style** reproduces CAKE's allocation rule, $P_l = \hat{H}_l \cdot V_l$ with the same floor, but not its cascading schedule or eviction indicator. **StreamingLLM** keeps 4 sink tokens and the most recent tokens with a uniform budget. **Full cache** is the upper bound.

V. Implementation

The library (about 1,400 lines of Python, plus 950 lines of scripts and 525 lines of tests) plugs into Hugging Face Transformers [20] through its attention-function and attention-mask registries, so the same code runs Qwen2 and Llama models without patching model files. One prefill is shared by all methods for a prompt; each method then prunes a copy of the cache and decodes greedily. Because layers keep different numbers of tokens, decoding uses each layer's own cache length, and the single new query needs no mask.

Correctness is checked by 63 automated tests, run before every experiment stage. They verify that window entropies and scores equal those computed from the exact eager attention matrix; that full-cache decoding matches Hugging Face's reference logits for both model families; that decoding from uneven per-layer caches equals decoding from the full cache with the dropped tokens masked; that every allocator spends exactly B and respects floors and caps; and that a

100% budget reproduces the full-cache output while equal entropies reproduce SnapKV token for token.

Experiments ran on Kaggle's free 2×T4 instances. A stage runner splits work so both GPUs finish together, stops cleanly before the 12-hour limit, and resumes without repeating finished rows. Every result row records the method, budget, score, prediction, per-layer budgets and entropies, timings and environment, and all figures regenerate from these files with one command.

VI. Experimental Setup

Models. Qwen2.5-3B-Instruct [18] (36 layers, 16 query and 2 KV heads) is the main model; Llama-3.2-3B-Instruct [19] (28 layers, 24 query and 8 KV heads) tests generalisation. Both run in fp16 with greedy decoding and batch size 1 on one T4 each.

Tasks. Six LongBench [16] tasks, one per category: Qasper (single-document QA), HotpotQA (multi-document QA), GovReport (summarisation), TREC (few-shot classification), PassageRetrieval-en (synthetic) and LCC (code), with the official prompts, generation lengths and metrics, and middle truncation to 8K tokens. Passkey retrieval [17] hides a 5-digit key at 0–100% depth in 4K, 8K and 16K tokens of filler text.

Budgets and statistics. Budgets are 10, 20, 30 and 50% of the full cache (2–5% for the hardest passkey runs). Every comparison is a paired difference on the same samples with a 95% bootstrap interval (1,000 resamples) [21]; we call a difference significant when its interval excludes zero. Table II lists the runs.

TABLE II
Experiments and sizes

Experiment	Model	Samples	Budgets	Rows
Main LongBench	Qwen	6 × 100	10/20/30/50%	17,400
Passkey heatmaps	Qwen	3 × 5 × 20	10, 20%	3,300
Passkey, extreme	Qwen	3 × 5 × 10	2, 5%	1,500
Ablations (τ, floor, seeds)	Qwen	6 × 50	10%	3,000
Window 16 / 64	Qwen	6 × 50	10%	1,200
Direction test	Both	1 × 5 × 30	2, 3%	2,400
Generalisation	Llama	6 × 50	10, 20%	3,900
Memory / latency	Qwen	3 lengths × 3	10–50%	153

Rows include the full-cache reference and all methods; 32,700 result rows in total.

VII. Results

A. RQ1: accuracy versus budget

Fig. 2 and Table III summarise the main experiment. Methods that split the budget evenly or nearly evenly (SnapKV, EntropyKV, its static variant and CAKE-style) form one tight group that keeps 91–92% of full-cache accuracy at 10%, 96–97% at 20% and 97–99% at 30%. PyramidKV and random allocation fall clearly below this group at 10–20%, and StreamingLLM, which cannot retrieve from the middle of the context, is worst everywhere.

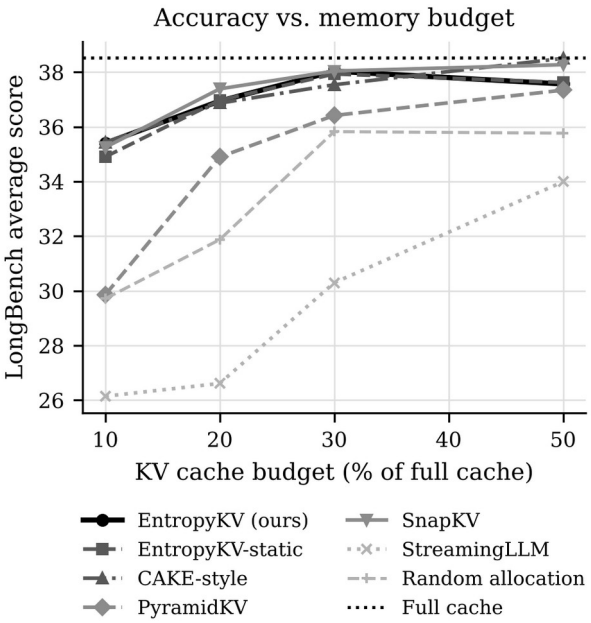


Fig. 2. Average LongBench score (six tasks, 600 samples) against KV budget for Qwen2.5-3B. Full cache scores 38.52.

TABLE III
LongBench average score (Qwen2.5-3B) by budget

Method	10%	20%	30%	50%
Full cache (100%)	38.52	38.52	38.52	38.52
EntropyKV (ours)	35.40	36.94	38.03	37.57
EntropyKV-static	34.90	36.97	37.93	37.62
SnapKV (uniform)	35.25	**37.39**	**38.04**	38.27
CAKE-style	**35.43**	36.87	37.54	**38.50**
PyramidKV	29.86	34.90	36.42	37.34
Random allocation	29.71	31.87	35.82	35.77
StreamingLLM	26.14	26.61	30.30	33.99

Scores ×100 on the official LongBench metrics; best pruned method per budget in bold.

TABLE IV
EntropyKV minus baseline (points, 95% CI)

Baseline	10%	20%	30%	50%
SnapKV	+0.15	−0.45†	−0.01	−0.71†
PyramidKV	+5.54†	+2.05†	+1.62†	+0.23
Random alloc.	+5.69†	+5.07†	+2.21†	+1.80†
StreamingLLM	+9.25†	+10.33†	+7.74†	+3.58†
EntropyKV-static	+0.50	−0.03	+0.10	−0.06
CAKE-style	−0.04	+0.08	+0.50	−0.94†

† 95% paired-bootstrap interval excludes zero (600 pairs). Example intervals: vs SnapKV at 20%, [−0.91, −0.02]; vs PyramidKV at 10%, [3.88, 7.44]; vs static at 10%, [−0.05, 1.13], p = 0.09.

Table IV gives the paired differences. EntropyKV beats PyramidKV by 5.5 points at 10%, 2.0 at 20% and 1.6 at 30%, and beats random allocation and StreamingLLM at every budget. Against the uniform split it does not win: it ties SnapKV at 10% and 30% and is slightly but significantly worse at 20% (−0.45) and 50% (−0.71). Per task (Table V), the largest differences between allocation rules appear on Qasper, LCC and TREC; among the near-even splits, HotpotQA and PassageRetrieval barely change.

TABLE V
PER-TASK SCORES AT 20% BUDGET (QWEN2.5-3B)

Method	Qasp.	Hotp.	GovR.	TREC	PRet.	LCC	Avg.
Full cache	34.49	44.52	18.57	65.0	39.0	29.52	38.52
EntropyKV	30.79	45.31	16.62	64.0	39.0	25.94	36.94
Ent.-static	30.49	45.31	16.98	63.0	39.0	27.05	36.97
SnapKV	32.33	45.64	16.59	64.0	39.0	26.76	37.39
CAKE-style	31.09	45.18	17.05	63.0	38.0	26.87	36.87
PyramidKV	26.96	45.51	15.56	59.0	40.0	22.35	34.90
Random	28.30	43.92	15.84	56.0	22.0	25.18	31.87
Streaming	20.17	36.98	15.63	52.0	12.0	22.90	26.61

B. Entropy profiles

Fig. 3 shows the per-layer entropy of Qwen2.5-3B for four inputs. The profile is not a pyramid: entropy is highest in layers 1–2 and 31–35, lowest around layers 5 and 25, and irregular in between. Inputs do differ (TREC's few-shot prompt raises entropy in several middle layers), but most of the shape is fixed by the model: over 600 inputs, the standard deviation of mean entropy *between* layers is 0.082, nearly twice the typical variation of one layer *across* inputs (0.044).

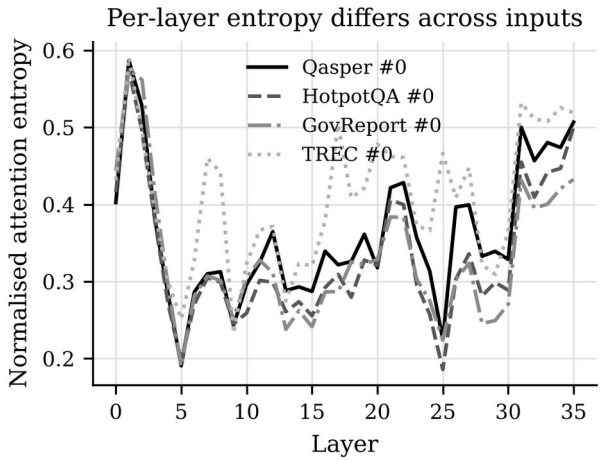


Fig. 3. Normalised attention entropy per layer for one prompt from each of four tasks (Qwen2.5-3B).

C. RQ2: dynamic versus static profile

Because most of the profile is model-determined, a frozen profile computed once from held-out prompts should capture most of what per-input recomputation captures, and it does. Across budgets the dynamic–static difference is +0.50, −0.03, +0.10 and −0.06 points, none significant (Table IV). Fig. 4 breaks the 20% result down by task: no task shows a reliable dynamic gain above one point, while the gain over PyramidKV is visible on Qasper, LCC and TREC.

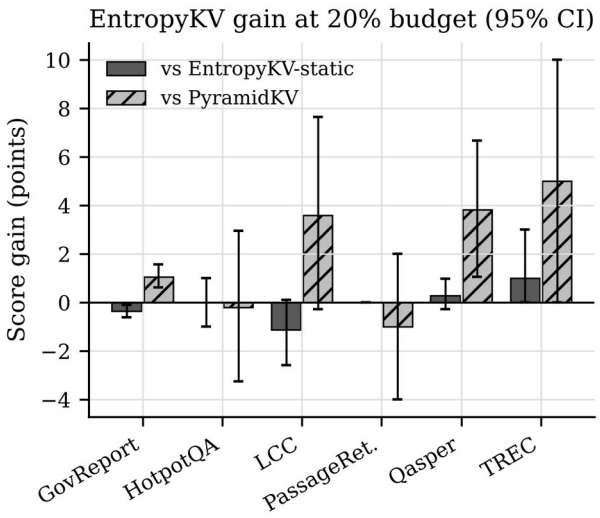


Fig. 4. EntropyKV gain per task at 20% budget over its frozen-profile variant (dark) and over PyramidKV (hatched), with 95% bootstrap intervals (100 samples per task).

D. RQ3: entropy alone versus CAKE-style

On LongBench, entropy alone matches the entropy×variance rule: the difference is within ±0.5 points and not significant at 10–30%, and −0.94 at 50% (borderline significant). EntropyKV reaches 99–101% of CAKE-style's score. The passkey results below are the exception.

E. Passkey retrieval

At 10% and 20% budgets every method except StreamingLLM retrieves the key in 100% of trials at all lengths and depths; StreamingLLM succeeds only when the key is in its recent window. The test only separates methods at extreme budgets (Fig. 5). At 2% of a 4K prompt, about 81 tokens per layer of which 32 are the observation window, CAKE-style retrieves 100% of keys, SnapKV 88%, EntropyKV 58%, random allocation 36% and PyramidKV 22%. With more absolute tokens per layer (8K and 16K prompts, or a 5% budget) nearly all methods return to 92–100%.

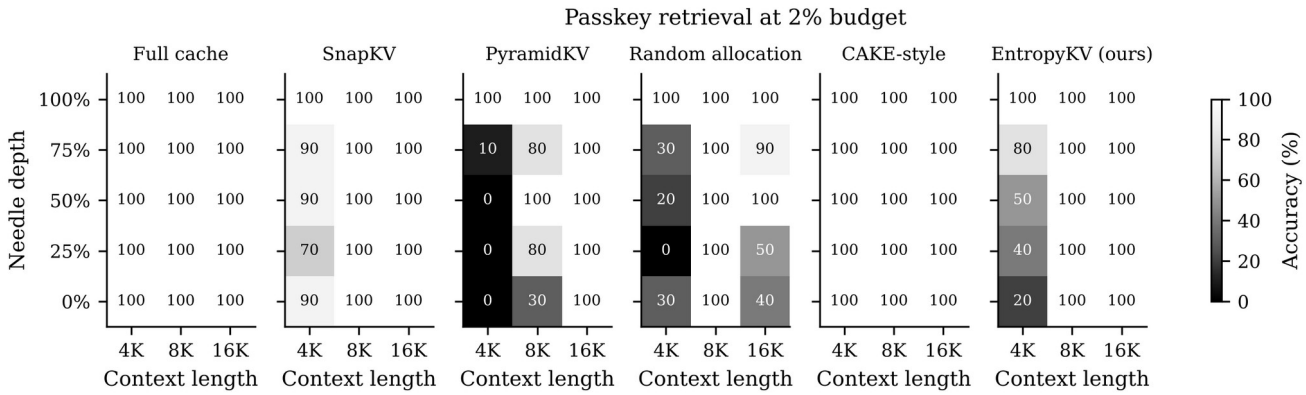


Fig. 5. Passkey retrieval accuracy at a 2% budget by context length and needle depth (Qwen2.5-3B, 10 trials per cell). Full cache is shown for reference.

F. Ablations

Fig. 6 compares each variant with SnapKV on the same 300 samples at 10% budget. Every entropy variant lies within ±0.5 points: temperature τ = 0.5, 1 or 2, floor 16 or 64, and observation windows of 16, 32 or 64 queries (the window size changes the absolute score of all methods, from 36.4 at w = 16 to 33.6 at w = 64 for SnapKV, but not the gap). Reversing the direction (τ = −1, more memory to focused layers) is as good as the default (+0.44). Random splits lose 0.9 to 4.9 points depending on the seed.

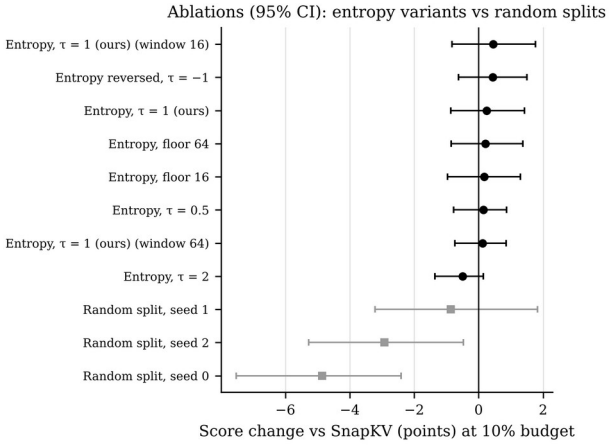


Fig. 6. Ablations at 10% budget (Qwen2.5-3B, 300 samples): score change against SnapKV with 95% paired-bootstrap intervals.

G. Why some allocations fail: layer starvation

If the direction of entropy does not matter, what separates the allocations that work from those that do not? Fig. 7 plots, for all 14 allocations at 10% budget, the score change against SnapKV versus the budget of the least-funded layer relative to an equal share. The correlation is r = 0.75. Every allocation whose smallest layer receives at most 0.10 of an equal share loses 0.9–5.5 points (PyramidKV gives its deepest layer 0.05; random splits 0.08–0.10). Every allocation that keeps its smallest layer above 0.15, including CAKE-style at 0.16 and all entropy variants at 0.34–0.77, stays within −0.50 to +0.44. On LongBench at these budgets, the useful property of entropy allocation is that it never starves a layer.

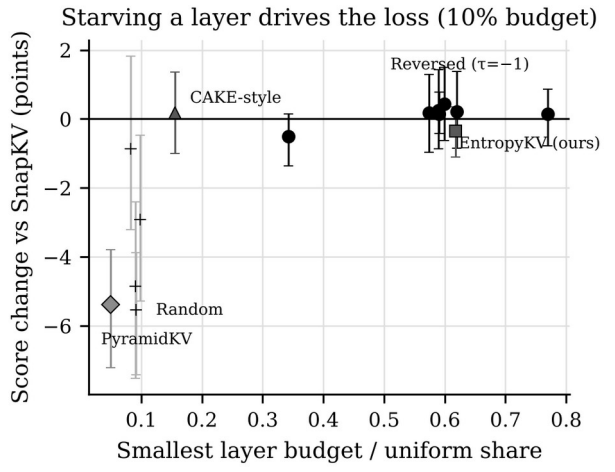


Fig. 7. Score change against SnapKV versus the smallest layer's budget relative to a uniform share, for 14 allocations at 10% budget. Crosses are random splits.

H. When direction matters: a targeted test

The 2% passkey failure contradicts the starvation account: EntropyKV's smallest layer still gets 0.71 of an equal share. Its per-layer budgets explain it instead. Qwen's lowest-entropy layers (focused attention, around layers 4–9) receive the fewest tokens from EntropyKV (58–65, against a mean of 81), whereas CAKE-style, whose variance term is large in the early-middle and middle layers, gives most of those layers 78–112 tokens and some middle layers up to 136, taking memory from the first and last layers instead. We hypothesised that these focused middle layers perform the retrieval, and tested it with a new run (150 trials per method and budget, 4K prompts): if the direction is the cause, reversing it should recover accuracy.

It does (Fig. 8). At 2%, reversed entropy (τ = −1) scores 88.7% against 64.7% for EntropyKV, a gain of 24.0 points (95% CI 16.0–31.3). CAKE-style reaches 97.3% and is the only rule to beat SnapKV significantly (+5.3, CI 0.7–9.3). At 3% every rule is at 99–100%, so the effect is confined to a narrow regime. On Llama-3.2-3B the same test shows no effect: all four rules retrieve 99.3–100% of keys at 2%.

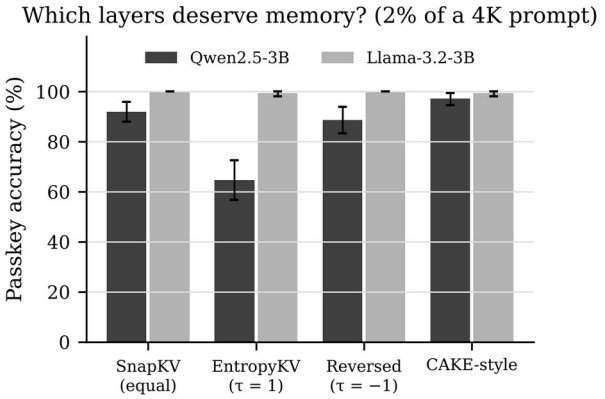


Fig. 8. Passkey accuracy at 2% of a 4K prompt for four allocation rules on both models (150 trials each, 95% bootstrap intervals).

I. Generalisation to Llama-3.2-3B

Table VI repeats the LongBench comparison on Llama-3.2-3B. The pattern holds: EntropyKV ties SnapKV (−0.27 at 10%, +0.18 at 20%, both not significant), ties its static variant, and keeps 94.5% and 98.3% of full-cache accuracy. PyramidKV's penalty is much smaller on Llama (−1.3 points at 10%, p = 0.05) even though its schedule starves the deepest layer just as much, so how costly starvation is depends on which layer and which model. No pairwise difference on Llama is significant.

TABLE VI
LONGBENCH AVERAGE ON LLAMA-3.2-3B (300 SAMPLES)

Method	10%	20%
Full cache (100%)	47.46	47.46
EntropyKV (ours)	44.86	**46.63**
EntropyKV-static	44.55	46.58
EntropyKV reversed (τ = −1)	43.88	46.50
SnapKV (uniform)	**45.14**	46.44
CAKE-style	45.07	46.10
PyramidKV	43.59	45.80

EntropyKV minus SnapKV: −0.27 [−1.15, 0.49] at 10%, +0.18 [−0.53, 0.95] at 20%.

J. RQ4: memory, latency and overhead

Fig. 9 shows the cost side for a 16K-token prompt. Because every pruned method keeps the same total, they use the same cache memory: 57 MB at 10% against 573 MB for the full cache (10× less), and decode at 44 ms per token against 139 ms (3.2× faster). The gain grows with context length: at 4K tokens decoding speeds up only from 50 to 43 ms per token, because the cache is then small next to the weights. Computing entropies, allocating and selecting takes 14–19 ms at 4K, 18–26 ms at 8K and 26–39 ms at 16K tokens (growing with the budget), about the same as SnapKV's selection and under 1% of the 2.0–12.2 s prefill. On the main LongBench run (6.2K tokens on average) decoding speeds up from 63.1 to 38.4 ms per token at 10%, for 11.7 ms of allocation against a 3.0 s prefill. Peak memory during prefill is unchanged, since pruning happens after it.

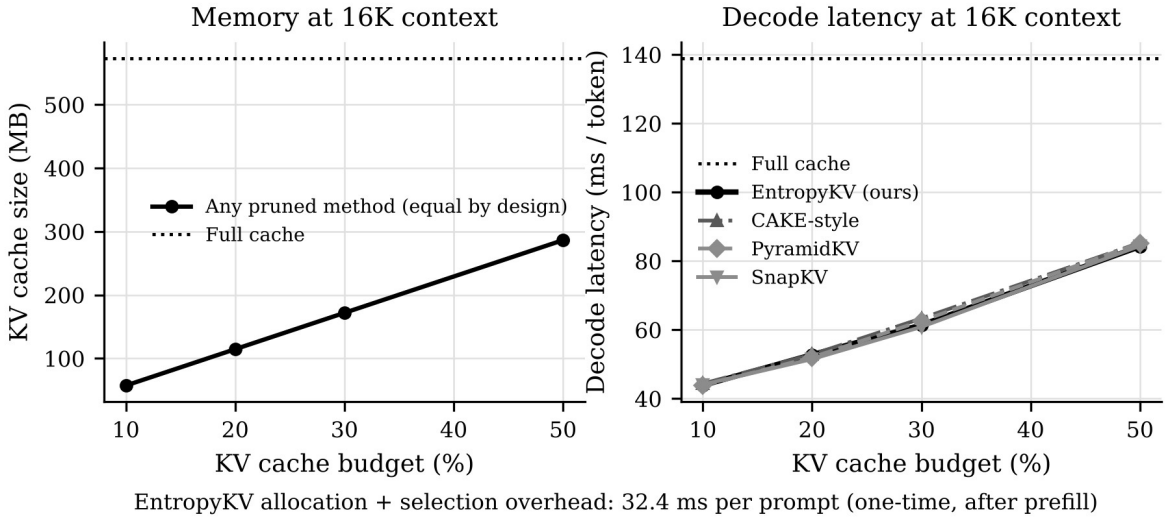


Fig. 9. KV-cache size (left) and decode latency (right) against budget for a 16K-token prompt on a T4 (Qwen2.5-3B, three repeats). All pruned methods keep the same number of tokens and therefore the same memory.

VIII. Discussion

What the hypotheses say. H1 is only partly supported: entropy allocation beats a fixed pyramid at 10–30%, but not a uniform split, and the gap to PyramidKV closes at 50% as predicted. H2 is rejected: no task shows a reliable dynamic-over-static gain, because the entropy profile is mostly a property of the model, not of the input. H3 is supported on LongBench, where entropy reaches 99–101% of CAKE-style, but not on extreme-budget retrieval for Qwen (66%).

What entropy allocation does and does not capture. The results are consistent with a simple picture. At moderate budgets, any split that gives every layer a reasonable share performs like the uniform split, and the main way to lose is to starve a layer, which a fixed pyramid or a random split does and entropy allocation does not. The direction of the signal then carries little information, which is why $\tau = -1$ ties $\tau = 1$ on LongBench. When the budget is so small that each layer holds little more than the observation window, the direction matters, and on Qwen "more memory to broad-attention layers" is the wrong direction for retrieval. This agrees with work showing that long-range retrieval is carried by a few heads with sharp attention [13], [14]: a focused, low-entropy layer may be one that is looking up something specific, not one that needs few tokens. CAKE's variance term happens to favour those layers, which explains its advantage here.

Why the two models differ. We can only offer a hypothesis. Qwen2.5-3B shares each KV head across eight query heads (2 KV heads over 36 layers), while Llama-3.2-3B uses 8 KV heads over 28 layers. With fewer KV heads, one token set per layer must serve more query heads, which may make Qwen more fragile when a retrieval-heavy layer is starved. Testing this needs more models than this project could run.

Practical recommendation. On small GQA models with budgets of 10% or more, a uniform split (SnapKV) is as good as any per-layer rule we tested and is simplest. If an adaptive rule is used, it should guarantee every layer a minimum share and should not assume that focused layers need less memory.

IX. Limitations

- **Prefill peak memory is unchanged.** Pruning happens after prefill, so savings are in the cache held during decoding and in decode latency. CAKE's cascading schedule addresses prefill peak; we did not implement it.
- **The question is inside the observation window during compression,** as in SnapKV, PyramidKV and CAKE. This favours window-scoring methods relative to a question-agnostic protocol such as kvpress [22].
- **Scale.** Two 3B models, contexts up to 8K tokens on LongBench and 16K on passkey, 50–100 samples per task. Intervals are about ±1 point, so smaller effects are not resolved.
- **Granularity.** One token set per layer, shared by its heads; head-level allocation (Ada-KV) is outside the study.
- **CAKE-style is a partial reproduction** of CAKE's allocation rule only, without its cascading memory management or eviction indicator, so it does not represent CAKE's full results.
- **Timings** were measured with our own attention backend in eager PyTorch on a T4; absolute numbers will differ with fused kernels, though ratios between methods should hold.

X. Conclusion and Future Work

We measured, under controlled conditions, how much per-input adaptivity of the per-layer KV budget buys on small GQA models. Entropy-driven allocation is cheap (under 1% of prefill time), keeps 92% of full-cache accuracy at a 10% budget, and clearly outperforms a fixed pyramid and random splits, while cutting cache memory 10× and decode latency 3.2× at 16K tokens. However, it does not beat a uniform split on LongBench, and recomputing the split per input adds nothing over a frozen profile. The variable that explains failures is layer starvation, and at extreme budgets the direction of the entropy signal can be wrong in a model-dependent way, which we confirmed by reversing it.

Future work: (i) cascading allocation during prefill to reduce peak memory; (ii) head-level budgets combined with retrieval-head detection, to test whether "focused but important" heads explain the direction effect; (iii) more models, especially varying the number of KV heads at fixed size, to test the hypothesis of Section VIII; and (iv) a question-agnostic evaluation protocol.

Team Contributions

TABLE VII
Contribution matrix

Member	Work
Khush Patel	Problem reformulation after DA1 review; framework design and implementation (attention backend, allocators, engine, runner); test suite; Kaggle pipeline; all experiment runs; analysis, figures and report.
Sneha Dash	[To be completed by the team: list only the work Sneha actually did.]

Acknowledgment

We thank Dr. Vijayaprabakaran K for guidance, and Kaggle for free GPU time. An AI assistant (Claude, Anthropic) was used for help with code, analysis and drafting; all experiments were run by the authors and every reported number was generated from the logged results in the public repository [23].

References

[1] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[2] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[3] J. Ainslie *et al.*, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, 2023.

[4] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Proc. NeurIPS*, 2023.

[5] Z. Liu *et al.*, "Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time," in *Proc. NeurIPS*, 2023.

[6] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. ICLR*, 2024.

[7] Y. Li *et al.*, "SnapKV: LLM knows what you are looking for before generation," in *Proc. NeurIPS*, 2024.

[8] Z. Cai *et al.*, "PyramidKV: Dynamic KV cache compression based on pyramidal information funneling," arXiv:2406.02069, 2024.

[9] X. Zhou *et al.*, "DynamicKV: Task-aware adaptive KV cache compression for long context LLMs," in *Findings of EMNLP*, 2025, arXiv:2412.14838.

[10] Z. Qin *et al.*, "CAKE: Cascading and adaptive KV cache eviction with layer preferences," in *Proc. ICLR*, 2025, arXiv:2503.12491.

[11] Y. Feng *et al.*, "Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference," arXiv:2407.11550, 2024.

[12] S. Ge *et al.*, "Model tells you what to discard: Adaptive KV cache compression for LLMs," in *Proc. ICLR*, 2024.

[13] G. Xiao *et al.*, "DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads," in *Proc. ICLR*, 2025.

[14] W. Wu *et al.*, "Retrieval head mechanistically explains long-context factuality," arXiv:2404.15574, 2024.

[15] Z. Liu *et al.*, "KIVI: A tuning-free asymmetric 2bit quantization for KV cache," in *Proc. ICML*, 2024.

[16] Y. Bai *et al.*, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proc. ACL*, 2024.

[17] A. Mohtashami and M. Jaggi, "Landmark attention: Random-access infinite context length for transformers," in *Proc. NeurIPS*, 2023.

[18] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[19] Llama Team, AI @ Meta, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[20] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020.

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[22] NVIDIA, "kvpress: LLM KV cache compression made easy," GitHub repository, 2024. [Online]. Available: https://github.com/NVIDIA/kvpress

[23] K. Patel and S. Dash, "kv-cache-entropy-pruning," GitHub repository, 2026. [Online]. Available: https://github.com/khushpatel3016/kv-cache-entropy-pruning